Now consider the command sequence, which will print numbers and segments starting from the first argument followed by the second.

```
user@host ~:$ seq 1 5
1
2
3
4
5
```

With the help of command expansion, it gets easy to instruct seq 1 5 into a possible variable by including this command as- $ (and) and then to pass it as an argument in another command at the result:

```
user@host~:$ echo $(seq 1 5)
1 2 3 4 5
# Or, to create 5 new directories:
user@host~:$ mkdir $(seq 1 5)
```

Single and double-quotes – Single quotes are used around the text so that it can prevent the shell from interpreting any other special characteristics. The special characters in it include dollar sign, asterisk, ampersand, spaces, and all the other characters which might get ignored when enclosed in single quotes. While handling the strings of text, this is an efficient way of preventing the shell interpretation and trying to substitute the variables.

Even if the semicolons are ignored they are placed between single quotes. As the new line character does not get interpreted between the single line quotes, the string within that can easily be made to sustain for multiple lines.

```
$ echo 'Text within single quotes can
> span multiple lines
> because the newline character
> is not interpereted'

Text within single quotes can
span multiple lines
because the newline character
is not interpreted
```